Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет
Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01 – «Информатика, вычислительная техника»

Дискретная математика и математическая логика
Лабораторная работа №1

Выполнил студент гр. ИВТ-24-1б:

Оглезнев Никита Михайлович
(Фамилия, Имя, Отчество)

(подпись)

Проверил:

ст. преп. каф. ИТАС
Рустамханова Г. И.
(должность, Ф.И.О руководителя от кафедры)

(оценка) (подпись)

(дата)

Пермь 2026

Содержание

Содержание 2
Цель 3
Задачи 3
Теоретический блок 4
Реализация 5
Заключение 12
Приложение А 13
Приложение Б 17

Цель

Изучение представления графов с помощью матрицы смежности и освоение алгоритмов обработки графов

Задачи

В рамках лабораторной работы необходимо:

1. Реализовать ввод матрицы смежности графа из файла, с клавиатуры.
2. Проверить матрицу на симметричность.
3. При необходимости выполнить симметризацию (приведение к неориентированному графу).
4. Найти компоненты связности графа.
5. Построить матрицу достижимости.
6. Организовать вывод результатов в удобной форме.

Теоретический блок

1. Матрица смежности

Граф с N вершинами представляется матрицей $A \in \{0,1\}^{N \times N}$, где:

$$A[i][j] = \begin{cases} 1, \text{если существует ребро } i \to j \\ 0, \quad \text{иначе} \end{cases}$$

Для ориентированного графа матрица может быть несимметричной, для неориентированного графа выполняется:$A[i][j] = A[j][i]$

2. Симметризация графа

Если граф ориентированный, его можно привести к неориентированному:

$$A`[i][j] \vee A[j][i].$$

Это означает, что ребро существует, если оно есть хотя бы в одном направлении.

3. Компоненты связности

Компонента связности –максимальное множество вершин, в котором между любой парой вершин существует путь. Для поиска используется: DFS (поиск в глубину) или BFS (поиск в ширину).

4. Матрица достижимости

Матрица достижимости R – матрица, описывающая наличие путей между всеми парами вершин: $R[i][j] = 1 \Leftrightarrow$ существует путь из i в j. Диагональ всегда равна 1.

Строится при помощи алгоритма Уоршелла:

$$R[i][j] = R[i][j] \vee (R[i][k] \wedge R[k][j])$$

Реализация

Программа построена как последовательная обработка графа, начиная с получения входных данных и заканчивая вычислением его ключевых характеристик. Вся логика работы программы разбита на этапы, каждый из которых соответствует отдельной задаче.

Работа программы начинается с получения матрицы смежности. Пользователю предоставляется выбор: загрузить данные из файла или ввести их вручную с клавиатуры.

При чтении из файла содержимое сначала считывается целиком, затем разбивается на отдельные элементы (токены), после чего происходит их проверка. Аналогичная проверка выполняется и при ручном вводе, но уже построчно, что позволяет локализовать ошибку и не заставляет пользователя вводить всю матрицу заново.

Ключевую роль играет функция *parse_tokens_to_matrix* (рис.1), совершающая валидацию: проверяется, что количество элементов равно N^2 и каждый элемент принадлежит множеству {0, 1}.

```
def parse_tokens_to_matrix(tokens: list[str]) -> list[list[int]]:
    if len(tokens) != N * N:
        raise ValueError(
            f"Ожидается {N * N} чисел (0 или 1), получено {len(tokens)}."
        )
    nums = []
    for t in tokens:
        if t not in ("0", "1"):
            raise ValueError(f"Недопустимый символ в матрице: {t!r} (нужны 0 и 1).")
        nums.append(int(t))
    # Разбиваем линейный список из N*N чисел на N строк по N элементов.
    return [nums[i * N : (i + 1) * N] for i in range(N)]
```

Рисунок 1 – Функция валидации элементов матрицы

После проверки линейный список преобразуется в двумерную структуру – матрицу смежности.

После получения матрицы программа анализирует её структуру, чтобы понять, является ли граф ориентированным. Это делается через проверку симметричности относительно главной диагонали (рис.2). Если для всех пар вершин выполняется условие A[i][j]=A[j][i], граф считается неориентированным.

Реализация оптимизирована: проверяется только верхняя половина матрицы (где j>i), поскольку остальные элементы либо уже проверены, либо тривиальны. Это уменьшает количество сравнений почти вдвое без потери корректности.

```
def is_symmetric(matrix: list[list[int]]) -> bool:
    for i in range(N):
        for j in range(i + 1, N):
            if matrix[i][j] != matrix[j][i]:
                return False
    return True
```

Рисунок 2 – Функция проверки симметричности матрицы

Если матрица несимметрична, программа явно сообщает, что граф ориентированный, и далее приводит его к неориентированному виду.

Для последующих алгоритмов (поиск компонент связности) требуется неориентированный граф, поэтому, если исходный граф ориентированный, выполняется его симметризация (рис.3).

Смысл операции заключается в объединении дуг: если между двумя вершинами есть хотя бы одна направленная связь, то в новой модели считается, что между ними есть неориентированное ребро.

На уровне реализации это выражается через логическую операцию «ИЛИ»:

$$A'[i][j]=A[i][j] \vee A[j][i]$$

Результат записывается симметрично, что гарантирует корректность дальнейших вычислений. Даже если исходная матрица уже была симметричной, этот шаг не нарушает её структуру, что делает алгоритм универсальным.

```python
def symmetrize(matrix: list[list[int]]) -> list[list[int]]:
    m = [[0] * N for _ in range(N)]
    for i in range(N):
        for j in range(N):
            # Ребро неориентированного графа: было ли хоть одно направление дуги.
            v = 1 if (matrix[i][j] or matrix[j][i]) else 0
            m[i][j] = v
            m[j][i] = v
    return m
```

Рисунок 3 – Функция симметризации матрицы

После приведения графа к неориентированному виду выполняется поиск компонент связности (рис.4).

Используется алгоритм поиска в глубину (DFS), реализованный в итеративной форме через стек. Такой подход выбран вместо рекурсии, чтобы избежать потенциальных проблем с переполнением стека вызовов.

Алгоритм работает следующим образом:

1. создаётся массив посещённых вершин;
2. далее последовательно перебираются все вершины графа;
3. если вершина ещё не посещена, запускается обход, который находит все вершины, достижимые из неё;
4. найденное множество образует одну компоненту связности.

Каждая компонента формируется независимо, что соответствует её математическому определению как максимального связного подграфа.

Дополнительно вершины внутри компоненты сортируются перед выводом, что делает результат более читаемым и однозначным.

```python
def connected_components(adj: list[list[int]]) -> list[list[int]]:
    visited = [False] * N
    components: list[list[int]] = []

    for start in range(N):
        if visited[start]:
            continue
        stack = [start]
        visited[start] = True
        comp: list[int] = []
        while stack:
            u = stack.pop()
            comp.append(u)
            # Соседи в неориентированном графе: все v с ребром (u,v).
            for v in range(N):
                if adj[u][v] and not visited[v]:
                    visited[v] = True
                    stack.append(v)
        components.append(sorted(comp))

    return components
```

Рисунок 4 – Функция поиска компонент связности через алгоритм обхода в глубину

На завершающем этапе программа определяет достижимость между всеми парами вершин, то есть строит транзитивное замыкание графа. Для этого используется алгоритм Уоршелла (рис. 5).

Сначала формируется начальная матрица:

- каждая вершина достижима из самой себя,
- учитываются все прямые рёбра графа.

Далее выполняется тройной цикл, в котором последовательно проверяется возможность пройти из вершины i в вершину j через промежуточную вершину k. Если такой путь существует, соответствующий элемент матрицы устанавливается в 1.

На каждом шаге алгоритм “разрешает” использовать новую вершину как промежуточную, постепенно расширяя множество возможных путей. После завершения всех итераций получается полная информация о достижимости. Для

связных компонент результатом становится подматрица, заполненная единицами, что соответствует полной достижимости внутри компоненты.

```python
def reachability_matrix(adj: list[list[int]]) -> list[list[int]]:
    r = [[0] * N for _ in range(N)]
    # Рефлексивно-замкнутый старт: петли и ребра длины 1.
    for i in range(N):
        for j in range(N):
            r[i][j] = 1 if (i == j or adj[i][j]) else 0

    # k — максимальный номер промежуточной вершины на текущем шаге (Уоршелл).
    for k in range(N):
        for i in range(N):
            for j in range(N):
                if r[i][k] and r[k][j]:
                    r[i][j] = 1

    return r
```

Рисунок 5 – Функция построения транзитивного замыкания графа при помощи алгоритма Уоршелла

Управление всей логикой осуществляется через функцию main (рис.6), которая реализует интерактивный цикл работы.

Она отвечает за:

- выбор источника данных,
- обработку ошибок ввода,
- последовательный вызов всех этапов обработки,
- вывод результатов пользователю.

Такое разделение делает программу модульной: каждая функция отвечает за свою часть логики, а основная функция лишь координирует их взаимодействие. Это упрощает как понимание кода, так и его возможное расширение.

```
def main():
    """Интерактивный цикл: выбор ввода → расчёты → вывод → снова меню."""
    base_dir = os.path.dirname(os.path.abspath(__file__))
    matrices_dir = os.path.join(base_dir, MATRICES_SUBDIR)
    default_name = "g11.txt"

    print("Матрица смежности графа (", N, "×", N, ")", sep="")

    while True:
        # --- Меню источника данных ---
        print()
        print("Источник данных:")
        print("  1 — файл")
        print("  2 — ручной ввод с клавиатуры")
        print("  0 — выход")
        choice = input("Выбор (0/1/2) [1]: ").strip() or "1"

        if choice == "0":
            print("Выход.")
            break
        if choice not in ("1", "2"):
            print("Неверный выбор. Повторите.")
            continue

        try:
            if choice == "1":
                if not os.path.isdir(matrices_dir):
                    print(f"Ошибка: папка с матрицами не найдена: {matrices_dir}")
                    continue
                names = matrix_filenames(matrices_dir)
                print(f"Папка с матрицами: {matrices_dir}")
                if names:
                    print("Файлы: " + ", ".join(names))
                else:
                    print("В папке нет .txt файлов.")
                prompt = f"Имя файла [{default_name}]: "
                name = input(prompt).strip() or default_name
                path = path_for_matrix_name(matrices_dir, name)
                matrix = read_matrix_from_file(path)
                print(f"\nПрочитано: {os.path.basename(path)}")
            else:
                matrix = read_matrix_manual()
        except (ValueError, FileNotFoundError) as e:
            print(f"Ошибка: {e}")
            continue

        # --- Шаг 1: исходная матрица и вывод о типе графа (ориент. / неориент.) ---
        print("\n--- Исходная матрица смежности ---")
        print_matrix(matrix)

        if is_symmetric(matrix):
            print(
```

```
                "Матрица смежности симметрична относительно главной диагонали.\n"
                "Следовательно, граф является неориентированным."
            )
        else:
            print(
                "Матрица смежности не симметрична относительно главной диагонали.\n"
                "Следовательно, граф ориентированный; для компонент связности и "
                "матрицы достижимости он приводится к неориентированному
(симметризация)."
            )
        print()

        # --- Шаг 2: симметризация; дальше все теоретические шаги — на
неориентированном графе ---
        sym = symmetrize(matrix)
        print("--- После приведения к неориентированному графу (симметризация) ---")
        if matrix == sym:
            print("(Матрица уже была симметричной — изменений нет.)\n")
        print_matrix(sym)

        # --- Шаг 3: компоненты связности по симметризованной матрице ---
        comps = connected_components(sym)
        print("--- Компоненты связности ---")
        print(f"Количество компонент связности: {len(comps)}")
        for idx, comp in enumerate(comps, start=1):
            verts = ", ".join(str(v + 1) for v in comp)
            print(f"  Компонента {idx}: вершины {{{verts}}} (нумерация с 1)")

        # --- Шаг 4: матрица достижимости для того же неориентированного графа ---
        reach = reachability_matrix(sym)
        print("--- Матрица достижимости (после симметризации) ---")
        print_matrix(reach)
```

Рисунок 6 – Основная функция

Заключение

В ходе выполнения лабораторной работы была реализована полноценная программа для анализа графа, заданного матрицей смежности, с последовательной обработкой всех ключевых характеристик его структуры.

На практике удалось проследить, как теоретические определения напрямую переходят в алгоритмы. Проверка симметричности матрицы естественным образом определяет тип графа, операция симметризации формализует переход от ориентированной модели к неориентированной, а поиск компонент связности через обход в глубину демонстрирует, как локальные переходы по рёбрам приводят к выделению глобальной структуры графа.

В рамках лабораторной работы были не только закреплены теоретические основы работы с графами, но и получен практический опыт их алгоритмической реализации, что является важным шагом в понимании методов дискретной математики и структур данных.

Приложение А

Результаты работы программы на тестовых файлах.

```
Имя файла [g11.txt]: test/g11.txt

Прочитано: g11.txt

--- Исходная матрица смежности ---
0 0 0 1 0 0 0 0 1 0
0 0 0 0 0 1 0 0 0 1
0 0 0 0 1 0 0 0 0 0
1 0 0 0 0 0 1 0 0 0
0 0 1 0 0 0 0 1 0 0
0 1 0 0 0 0 0 0 0 1
0 0 0 1 0 0 0 0 1 0
0 0 0 0 1 0 0 0 0 0
1 0 0 0 0 0 1 0 0 0
0 1 0 0 0 1 0 0 0 0

Матрица смежности симметрична относительно главной диагонали.
Следовательно, граф является неориентированным.

--- После приведения к неориентированному графу (симметризация) ---
(Матрица уже была симметричной — изменений нет.)

0 0 0 1 0 0 0 0 1 0
0 0 0 0 0 1 0 0 0 1
0 0 0 0 1 0 0 0 0 0
1 0 0 0 0 0 1 0 0 0
0 0 1 0 0 0 0 1 0 0
0 1 0 0 0 0 0 0 0 1
0 0 0 1 0 0 0 0 1 0
0 0 0 0 1 0 0 0 0 0
1 0 0 0 0 0 1 0 0 0
0 1 0 0 0 1 0 0 0 0

--- Компоненты связности ---
Количество компонент связности: 3
  Компонента 1: вершины {1, 4, 7, 9} (нумерация с 1)
  Компонента 2: вершины {2, 6, 10} (нумерация с 1)
  Компонента 3: вершины {3, 5, 8} (нумерация с 1)
--- Матрица достижимости (после симметризации) ---
1 0 0 1 0 0 1 0 1 0
0 1 0 0 0 1 0 0 0 1
0 0 1 0 1 0 0 1 0 0
1 0 0 1 0 0 1 0 1 0
0 0 1 0 1 0 0 1 0 0
0 1 0 0 0 1 0 0 0 1
1 0 0 1 0 0 1 0 1 0
0 0 1 0 1 0 0 1 0 0
1 0 0 1 0 0 1 0 1 0
0 1 0 0 0 1 0 0 0 1
```

```
Прочитано: g12.txt

--- Исходная матрица смежности ---
0 0 0 0 0 0 0 0 1 0
0 0 1 0 0 1 0 0 0 0
0 1 0 0 0 1 0 0 0 0
0 0 0 0 1 0 0 0 0 0
0 0 0 1 0 0 0 1 0 0
0 1 1 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 0
1 0 0 0 0 1 0 0 0 1
0 0 0 0 0 0 0 0 1 0

Матрица смежности симметрична относительно главной диагонали.
Следовательно, граф является неориентированным.

--- После приведения к неориентированному графу (симметризация) ---
(Матрица уже была симметричной — изменений нет.)

0 0 0 0 0 0 0 0 1 0
0 0 1 0 0 1 0 0 0 0
0 1 0 0 0 1 0 0 0 0
0 0 0 0 1 0 0 0 0 0
0 0 0 1 0 0 0 1 0 0
0 1 1 0 0 0 0 0 1 0
0 0 0 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0 0 0
1 0 0 0 0 1 0 0 0 1
0 0 0 0 0 0 0 0 1 0

--- Компоненты связности ---
Количество компонент связности: 3
  Компонента 1: вершины {1, 2, 3, 6, 9, 10} (нумерация с 1)
  Компонента 2: вершины {4, 5, 8} (нумерация с 1)
  Компонента 3: вершины {7} (нумерация с 1)
--- Матрица достижимости (после симметризации) ---
1 1 1 0 0 1 0 0 1 1
1 1 1 0 0 1 0 0 1 1
1 1 1 0 0 1 0 0 1 1
0 0 0 1 1 0 0 1 0 0
0 0 0 1 1 0 0 1 0 0
1 1 1 0 0 1 0 0 1 1
0 0 0 0 0 0 1 0 0 0
0 0 0 1 1 0 0 1 0 0
1 1 1 0 0 1 0 0 1 1
1 1 1 0 0 1 0 0 1 1
```

```
Прочитано: g13.txt

--- Исходная матрица смежности ---
0 0 0 0 0 0 1 1 1 0
0 0 0 0 0 1 0 0 0 1
0 0 0 1 0 0 0 0 0 1
0 0 1 0 1 0 0 0 0 0
0 0 0 1 0 1 0 0 0 0
0 1 0 0 1 0 0 0 0 0
1 0 0 0 0 0 0 1 1 0
1 0 0 0 0 0 1 0 1 0
1 0 0 0 0 0 1 1 0 0
0 1 1 0 0 0 0 0 0 0

Матрица смежности симметрична относительно главной диагонали.
Следовательно, граф является неориентированным.

--- После приведения к неориентированному графу (симметризация) ---
(Матрица уже была симметричной — изменений нет.)

0 0 0 0 0 0 1 1 1 0
0 0 0 0 0 1 0 0 0 1
0 0 0 1 0 0 0 0 0 1
0 0 1 0 1 0 0 0 0 0
0 0 0 1 0 1 0 0 0 0
0 1 0 0 1 0 0 0 0 0
1 0 0 0 0 0 0 1 1 0
1 0 0 0 0 0 1 0 1 0
1 0 0 0 0 0 1 1 0 0
0 1 1 0 0 0 0 0 0 0

--- Компоненты связности ---
Количество компонент связности: 2
  Компонента 1: вершины {1, 7, 8, 9} (нумерация с 1)
  Компонента 2: вершины {2, 3, 4, 5, 6, 10} (нумерация с 1)
--- Матрица достижимости (после симметризации) ---
1 0 0 0 0 0 1 1 1 0
0 1 1 1 1 1 0 0 0 1
0 1 1 1 1 1 0 0 0 1
0 1 1 1 1 1 0 0 0 1
0 1 1 1 1 1 0 0 0 1
0 1 1 1 1 1 0 0 0 1
1 0 0 0 0 0 1 1 1 0
1 0 0 0 0 0 1 1 1 0
1 0 0 0 0 0 1 1 1 0
0 1 1 1 1 1 0 0 0 1
```

```
Прочитано: g14.txt

--- Исходная матрица смежности ---
0 0 0 0 0 1 0 0 0 0
0 0 0 0 1 0 0 0 1 0
0 0 0 0 0 0 1 1 0 0
0 0 0 0 0 0 0 1 0 1
0 1 0 0 0 0 0 0 0 0
1 0 0 0 0 0 1 0 0 0
0 0 1 0 0 1 0 0 1 1
0 0 1 1 0 0 0 0 0 0
0 1 0 0 0 0 1 0 0 0
0 0 0 1 0 0 1 0 0 0

Матрица смежности симметрична относительно главной диагонали.
Следовательно, граф является неориентированным.

--- После приведения к неориентированному графу (симметризация) ---
(Матрица уже была симметричной — изменений нет.)

0 0 0 0 0 1 0 0 0 0
0 0 0 0 1 0 0 0 1 0
0 0 0 0 0 0 1 1 0 0
0 0 0 0 0 0 0 1 0 1
0 1 0 0 0 0 0 0 0 0
1 0 0 0 0 0 1 0 0 0
0 0 1 0 0 1 0 0 1 1
0 0 1 1 0 0 0 0 0 0
0 1 0 0 0 0 1 0 0 0
0 0 0 1 0 0 1 0 0 0

--- Компоненты связности ---
Количество компонент связности: 1
  Компонента 1: вершины {1, 2, 3, 4, 5, 6, 7, 8, 9, 10} (нумерация с 1)
--- Матрица достижимости (после симметризации) ---
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1
```

Приложение Б

Полный код программы.

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
Задаётся матрица смежности A размера N×N с элементами 0 и 1:
  A[i][j] = 1  — есть ребро (дуга) из вершины i в вершину j,
  A[i][j] = 0  — ребра нет.
Вершины нумеруются индексами 0..N-1 в коде и 1..N в выводе пользователю.

Дальше по условию лабораторной:
  • матрица при необходимости симметризуется (ориентированный граф заменяется
    неориентированным «объединением» парных дуг);
  • на неориентированном графе ищутся компоненты связности (обход графа);
  • строится матрица достижимости (транзитивное замыкание отношения смежности).
"""

import os

# Число вершин графа (размер матрицы смежности).
N = 10

MATRICES_SUBDIR = "test"


def matrix_filenames(matrices_dir: str) -> list[str]:
    """
    Служебная функция: собирает имена .txt файлов в папке с тестовыми матрицами.

    К дискретной математике не относится — только удобство выбора файла по имени.
    """
    if not os.path.isdir(matrices_dir):
        return []
    return sorted(f for f in os.listdir(matrices_dir) if f.lower().endswith(".txt"))


def path_for_matrix_name(matrices_dir: str, name: str) -> str:
    """
    Собирает полный путь к файлу матрицы только из имени файла.

    Берётся os.path.basename(name), чтобы пользователь не мог ввести путь вида
    «../другая_папка/файл» — чтение только из фиксированной папки matrices_dir.
    """
    safe = os.path.basename(name.strip())
    return os.path.join(matrices_dir, safe)


def print_matrix(matrix: list[list[int]], title: str | None = None) -> None:
    """Выводит матрицу построчно (элементы через пробел)."""
    if title:
```

```
        print(title)
    for row in matrix:
        print(" ".join(str(x) for x in row))
    print()


def parse_tokens_to_matrix(tokens: list[str]) -> list[list[int]]:
    """
    Превращает ровно N×N токенов «0» или «1» в список из N строк матрицы.

    Матрица смежности кодирует граф: строка i — исходящие рёбра из вершины i.
    Диагональ обычно 0 (петель нет), но условие задаёт только 0/1 — петли
    допускаются, если так записано во входе.
    """
    if len(tokens) != N * N:
        raise ValueError(
            f"Ожидается {N * N} чисел (0 или 1), получено {len(tokens)}."
        )
    nums = []
    for t in tokens:
        if t not in ("0", "1"):
            raise ValueError(f"Недопустимый символ в матрице: {t!r} (нужны 0 и 1).")
        nums.append(int(t))
    # Разбиваем линейный список из N*N чисел на N строк по N элементов.
    return [nums[i * N : (i + 1) * N] for i in range(N)]


def read_matrix_from_file(path: str) -> list[list[int]]:
    """Читает весь файл, режет по пробельным символам, парсит как матрицу N×N."""
    path = os.path.expanduser(path)
    if not os.path.isfile(path):
        raise FileNotFoundError(f"Файл не найден: {path}")
    with open(path, "r", encoding="utf-8") as f:
        text = f.read()
    tokens = text.split()
    return parse_tokens_to_matrix(tokens)


def read_matrix_manual() -> list[list[int]]:
    """
    Пошаговый ввод матрицы с клавиатуры: N строк, в каждой ровно N чисел 0/1.

    Каждая строка ввода проверяется через parse_tokens_to_matrix отдельно —
    так проще сообщать об ошибке только для текущей строки.
    """
    print(f"Введите матрицу {N}×{N} построчно: в каждой строке {N} чисел 0 или 1 через
пробел.")
    rows: list[list[int]] = []
    for i in range(N):
        while True:
            line = input(f"Строка {i + 1}/{N}: ").strip()
            parts = line.split()
            try:
```

```
                row = parse_tokens_to_matrix(parts)[0]
            except ValueError as e:
                print(e, "Повторите ввод.")
                continue
            rows.append(row)
            break
    return rows


def is_symmetric(matrix: list[list[int]]) -> bool:
    """
    Проверка симметрии матрицы смежности относительно главной диагонали.

    В теории графов: если A[i][j] = A[j][i] для всех i, j, то матрица смежности
    задаёт неориентированный граф (ребро {i,j} кодируется одинаково в обе стороны).

    Достаточно сравнить пары (i, j) при j > i — остальное следует из уже
    проверенных пар и диагонали (на диагонали i=j сравнение с самой собой тривиально).
    """
    for i in range(N):
        for j in range(i + 1, N):
            if matrix[i][j] != matrix[j][i]:
                return False
    return True


def symmetrize(matrix: list[list[int]]) -> list[list[int]]:
    """
    Переход от возможного ориентированного графа к неориентированному по матрице
смежности.

    Дискретная математика:
      Если дуга есть хотя бы в одном направлении (i→j или j→i), в неориентированном
      графе считаем, что есть неориентированное ребро между i и j.

    Формула для новой матрицы A':
      A'[i][j] = A'[j][i] = 1, если A[i][j] = 1 или A[j][i] = 1, иначе 0.
    Это то же самое, что взять покомпонентное «ИЛИ» для пары (A[i][j], A[j][i])
    и записать результат симметрично — получается симметричная матрица смежности.

    Результат всегда симметричен, по нему удобно считать компоненты связности
    как в неориентированном графе.
    """
    m = [[0] * N for _ in range(N)]
    for i in range(N):
        for j in range(N):
            # Ребро неориентированного графа: было ли хоть одно направление дуги.
            v = 1 if (matrix[i][j] or matrix[j][i]) else 0
            m[i][j] = v
            m[j][i] = v
    return m
```

```python
def connected_components(adj: list[list[int]]) -> list[list[int]]:
    """
    Компоненты связности неориентированного графа, заданного симметричной матрицей
adj.

    Дискретная математика:
      Компонента связности — максимальное по включению множество вершин, в котором
      любые две вершины соединены цепью рёбер (в неориентированном графе).

    Алгоритм: обход в глубину (DFS) из каждой ещё не посещённой вершины.
      • Внешний цикл for start перебирает стартовые вершины — так мы находим все
        компоненты: каждая новая «незакрашенная» вершина начинает новую компоненту.
      • Стек реализует DFS: из текущей вершины уходим в первого непосещённого соседа
        по матрице (adj[u][v] == 1).

    Возвращается список компонент; вершины внутри компоненты сортируются по номеру
    для аккуратного вывода.
    """
    visited = [False] * N
    components: list[list[int]] = []

    for start in range(N):
        if visited[start]:
            continue
        stack = [start]
        visited[start] = True
        comp: list[int] = []
        while stack:
            u = stack.pop()
            comp.append(u)
            # Соседи в неориентированном графе: все v с ребром (u,v).
            for v in range(N):
                if adj[u][v] and not visited[v]:
                    visited[v] = True
                    stack.append(v)
        components.append(sorted(comp))

    return components


def reachability_matrix(adj: list[list[int]]) -> list[list[int]]:
    """
    Матрица достижимости R (транзитивное замыкание бинарного отношения «есть ребро»).

    Дискретная математика:
      R[i][j] = 1, если существует путь из i в j (длина пути ≥ 0).
      Путь нулевой длины: вершина достижима сама из себя — поэтому на диагонали
      всегда 1 (рефлексивное замыкание).

      Для первого шага берём не только рёбра графа, но и «стояние на месте»:
      начальная матрица: R₀[i][j] = 1, если i = j или adj[i][j] = 1.
```

```
    Дальше — алгоритм Уоршелла (булевый вариант алгоритма Флойда — Уоршелла):
      для каждой k от 0 до N-1 разрешаем в качестве промежуточных вершин только
      номера ≤ k. Инвариант после шага k:
        R[i][j] = 1  ⇔  существует путь из i в j, все внутренние вершины которого
        имеют номера не больше k.

      Рекуррентное соотношение (логическое ИЛИ):
        R_new[i][j] = R[i][j] ∨ (R[i][k] ∧ R[k][j])
      То есть путь либо уже был без k, либо идёт через k: i → … → k → … → j.

    После N итераций учтены все вершины как промежуточные — получена полная
    достижимость. Для неориентированного связного куска матрица на этом куске
    будет «все единицы»: из любой вершины достижима любая другая.
    """
    r = [[0] * N for _ in range(N)]
    # Рефлексивно-замкнутый старт: петли и ребра длины 1.
    for i in range(N):
        for j in range(N):
            r[i][j] = 1 if (i == j or adj[i][j]) else 0

    # k — максимальный номер промежуточной вершины на текущем шаге (Уоршелл).
    for k in range(N):
        for i in range(N):
            for j in range(N):
                if r[i][k] and r[k][j]:
                    r[i][j] = 1

    return r


def main() -> None:
    """Интерактивный цикл: выбор ввода → расчёты → вывод → снова меню."""
    base_dir = os.path.dirname(os.path.abspath(__file__))
    matrices_dir = os.path.join(base_dir, MATRICES_SUBDIR)
    default_name = "g11.txt"

    print("Матрица смежности графа (", N, "×", N, ")", sep="")

    while True:
        # --- Меню источника данных ---
        print()
        print("Источник данных:")
        print("  1 — файл")
        print("  2 — ручной ввод с клавиатуры")
        print("  0 — выход")
        choice = input("Выбор (0/1/2) [1]: ").strip() or "1"

        if choice == "0":
            print("Выход.")
            break
        if choice not in ("1", "2"):
            print("Неверный выбор. Повторите.")
```

```
            continue

        try:
            if choice == "1":
                if not os.path.isdir(matrices_dir):
                    print(f"Ошибка: папка с матрицами не найдена: {matrices_dir}")
                    continue
                names = matrix_filenames(matrices_dir)
                print(f"Папка с матрицами: {matrices_dir}")
                if names:
                    print("Файлы: " + ", ".join(names))
                else:
                    print("В папке нет .txt файлов.")
                prompt = f"Имя файла [{default_name}]: "
                name = input(prompt).strip() or default_name
                path = path_for_matrix_name(matrices_dir, name)
                matrix = read_matrix_from_file(path)
                print(f"\nПрочитано: {os.path.basename(path)}")
            else:
                matrix = read_matrix_manual()
        except (ValueError, FileNotFoundError) as e:
            print(f"Ошибка: {e}")
            continue

        # --- Шаг 1: исходная матрица и вывод о типе графа (ориент. / неориент.) ---
        print("\n--- Исходная матрица смежности ---")
        print_matrix(matrix)

        if is_symmetric(matrix):
            print(
                "Матрица смежности симметрична относительно главной диагонали.\n"
                "Следовательно, граф является неориентированным."
            )
        else:
            print(
                "Матрица смежности не симметрична относительно главной диагонали.\n"
                "Следовательно, граф ориентированный; для компонент связности и "
                "матрицы достижимости он приводится к неориентированному
(симметризация)."
            )
        print()

        # --- Шаг 2: симметризация; дальше все теоретические шаги — на
неориентированном графе ---
        sym = symmetrize(matrix)
        print("--- После приведения к неориентированному графу (симметризация) ---")
        if matrix == sym:
            print("(Матрица уже была симметричной — изменений нет.)\n")
        print_matrix(sym)

        # --- Шаг 3: компоненты связности по симметризованной матрице ---
        comps = connected_components(sym)
```

```
        print("--- Компоненты связности ---")
        print(f"Количество компонент связности: {len(comps)}")
        for idx, comp in enumerate(comps, start=1):
            verts = ", ".join(str(v + 1) for v in comp)
            print(f"  Компонента {idx}: вершины {{{verts}}} (нумерация с 1)")

        # --- Шаг 4: матрица достижимости для того же неориентированного графа ---
        reach = reachability_matrix(sym)
        print("--- Матрица достижимости (после симметризации) ---")
        print_matrix(reach)


if __name__ == "__main__":
    main()
```